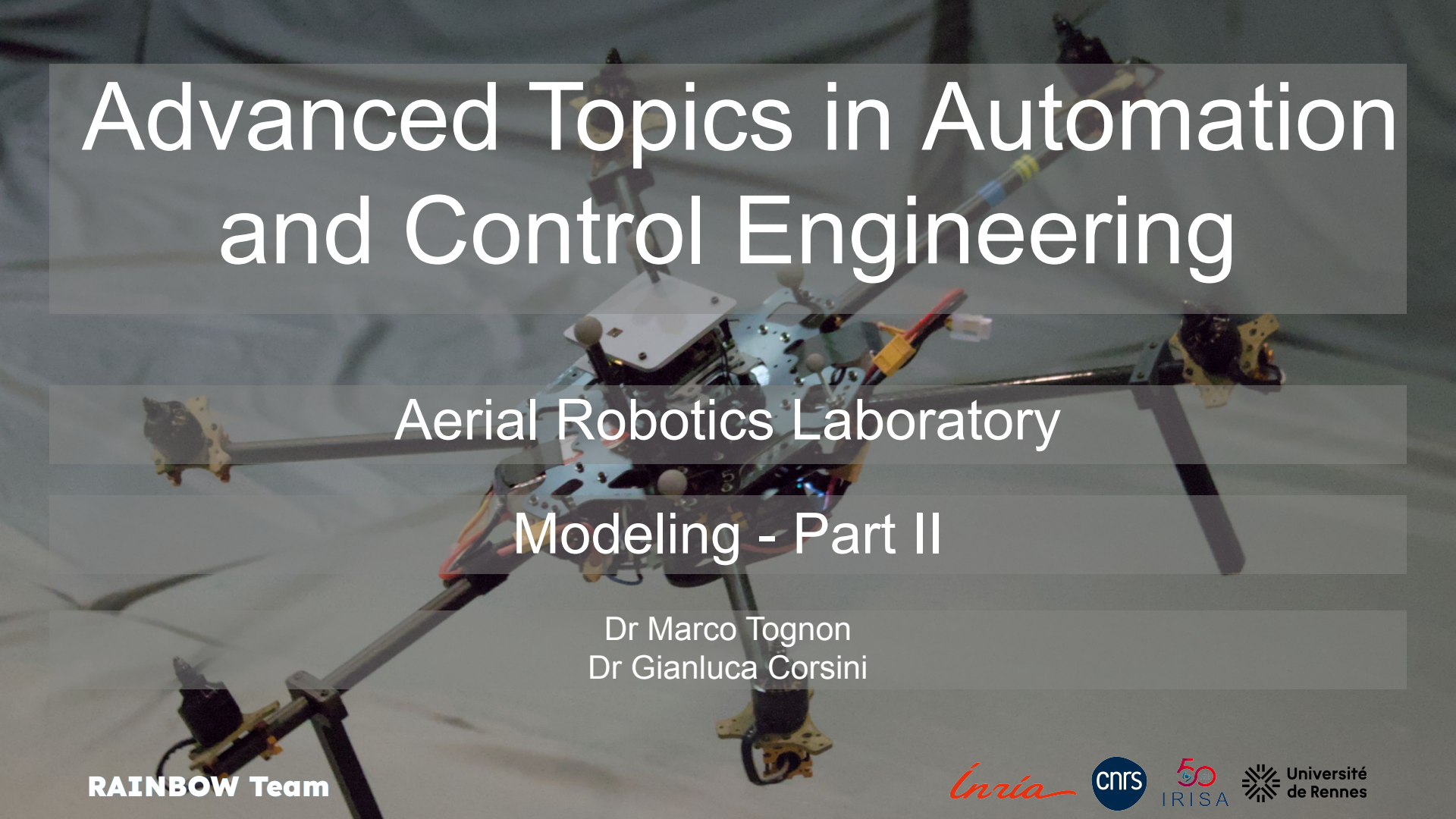
Advanced Topics in Automation and Control Engineering

Aerial Robotics Laboratory

Modeling - Part II

Dr Marco Tognon
Dr Gianluca Corsini



RAINBOW Team

Inria cnrs 50 IRISA Université de Rennes


A quick recap first …

Yesterday assignment

1. Run Gazebo simulations for the following robots
   a. Under-actuated quadrotor
   b. Under-actuated hexarotor
   c. Fully-actuated hexarotor
   d. [Under-actuated octorotor]
   e. [Fully-actuated octorotor]

2. Realize the following plots for each Gazebo simulation
   a. Tracking plots (desired vs measured) of i) position [m/s], ii) attitude (Euler) [deg], iii) linear velocities [m/s], iv) angular velocities [deg/s], v) linear accelerations [m/s^2]
   b. Controller errors for i) position [m], ii) attitude errors [deg], iii) linear velocities [m/s], iv) angular velocities [deg/s]
   c. Controller force [N] and torque [Nm] inputs sent to the robot

Yesterday assignment

Waypoints (wp) that each robot shall reach during the simulations:

1. Robot starts in wp0: `x=0, y=0, z=0, yaw=0`
2. After 2s from simulation start, ask for wp1: `x=1, y=1, z=1, yaw=0`
3. After 5s from previous wp, ask for wp2: `x=1, y=-1, z=1, yaw=pi/2`
4. After 5s from previous wp, ask for wp3 (end): `x=0, y=0, z=0, yaw =0`

TIP: Create function "`simulation`" that executes automatically all required functions for the simulation (setup, start, waypoints, …) when running the python script for a given robot.

Yesterday assignment

Recommended plot layouts and styles.

- Each table is a figure window, and each cell a subplot
- Measured values (current) plotted with continuous lines
- Desired values (reference) plotted with dashed lines
- Use RGB for XYZ axes, respectively
- Set limits along plot y axes (ymin, ymax)
- Start plots from time t=0

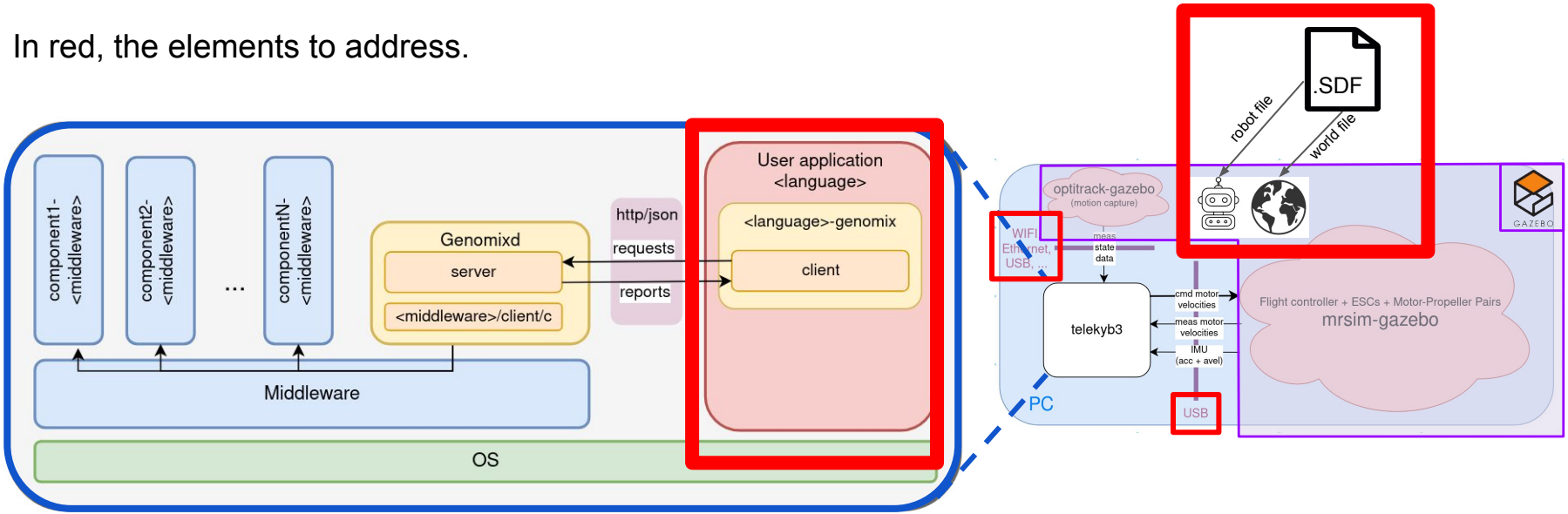
pos (x,y,z)	att (roll,pitch,yaw)
vel (v_x,v_y,v_z)	avel (w_x, w_y, w_z)
acc (a_x, a_y, a_z)	

e_pos (e_x, e_y, e_z)	e_att (e_roll, e_pitch, e_yaw)
e_vel (e_vx, e_vy, e_vz)	e_avel (e_wx, e_wy, e_wz)

f_props for each i in [1, n_props]

Yesterday assignment – Visual representation

In red, the elements to address.

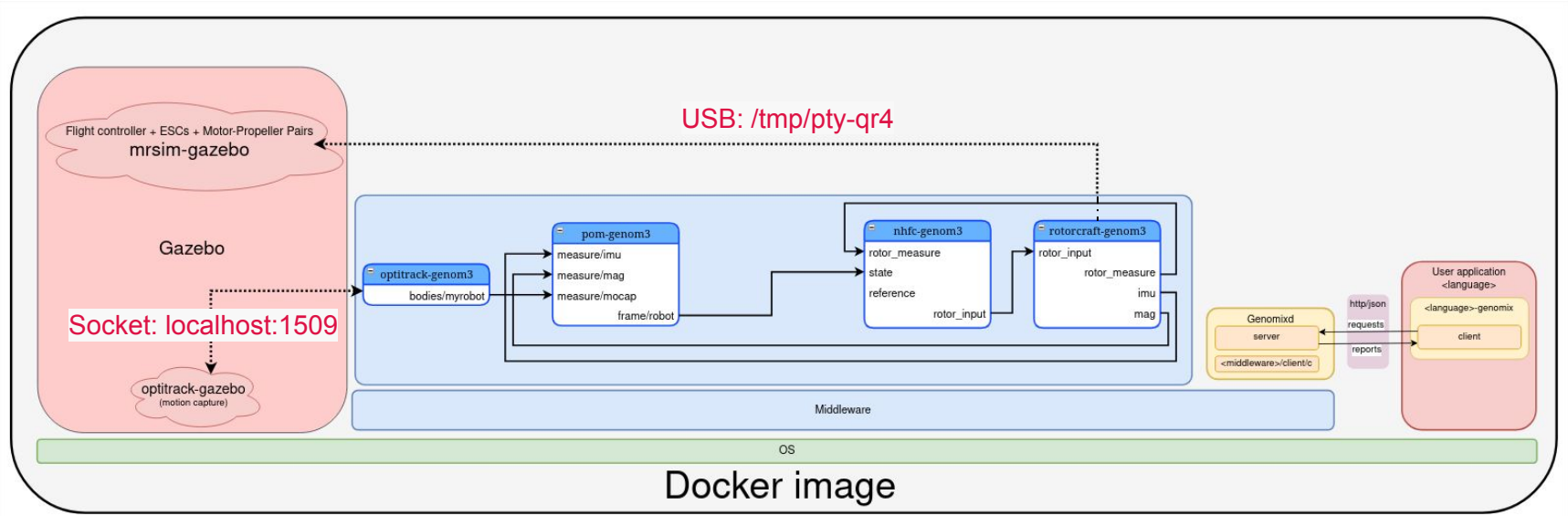


where

- *<middleware> = pocoLibs*
- *<language> = python (python-genomix documentation)*

telekyb3 – Quadrotor simulation in Gazebo

Overview with all the running GenoM3 components and the required port connections



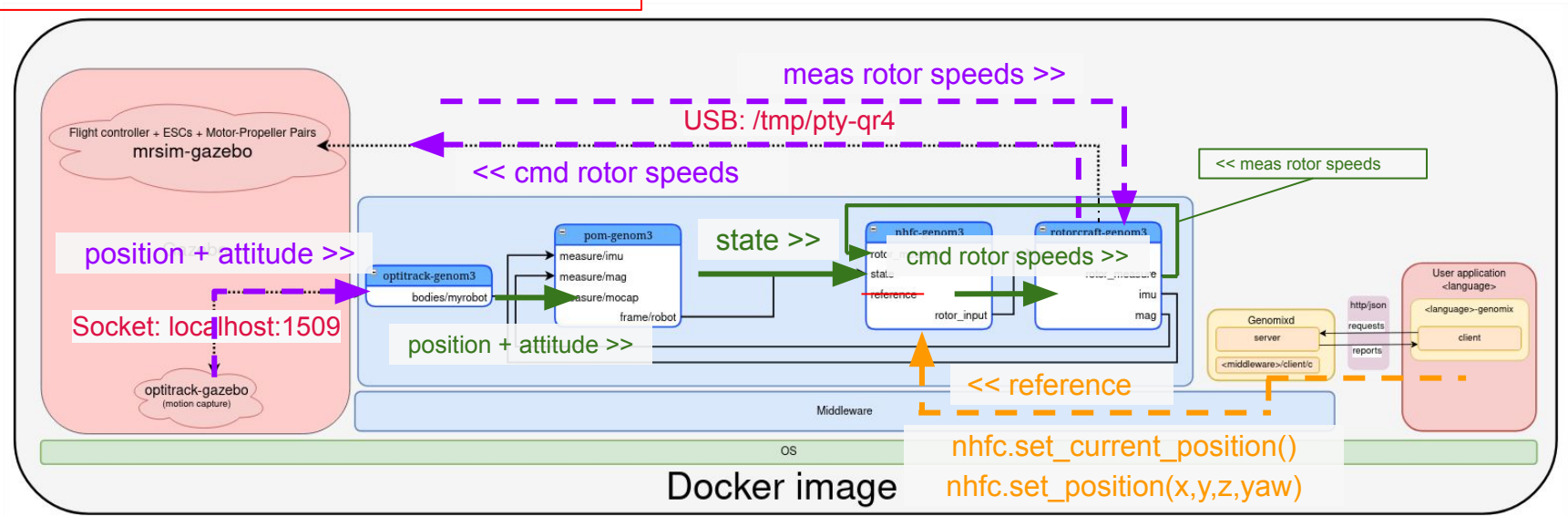
where

- <middleware> = pocoLibs

- <language> = python (python-genomix documentation)

telekyb3 – Quadrotor simulation in Gazebo

Visual representation of the data flow



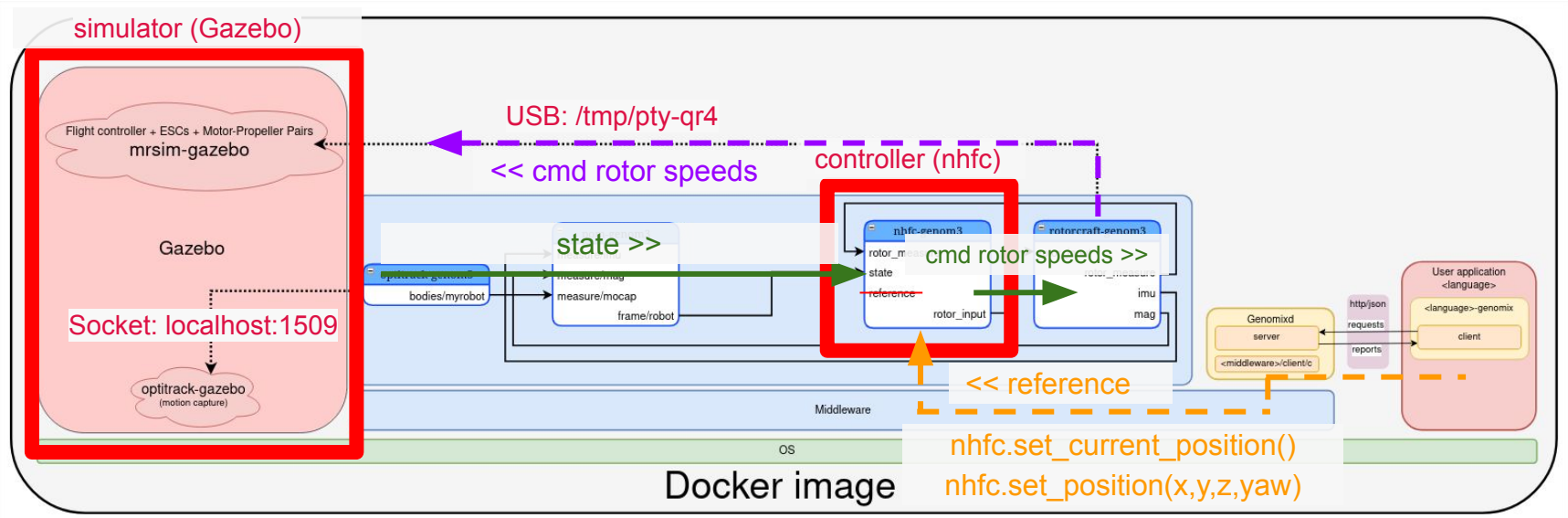
where
- *<middleware> = pocoLibs*
- *<language> = python (python-genomix documentation)*

- state := position, attitude, linear and angular velocities, linear and angular accelerations


Today…

telekyb3 – Quadrotor simulation in Gazebo

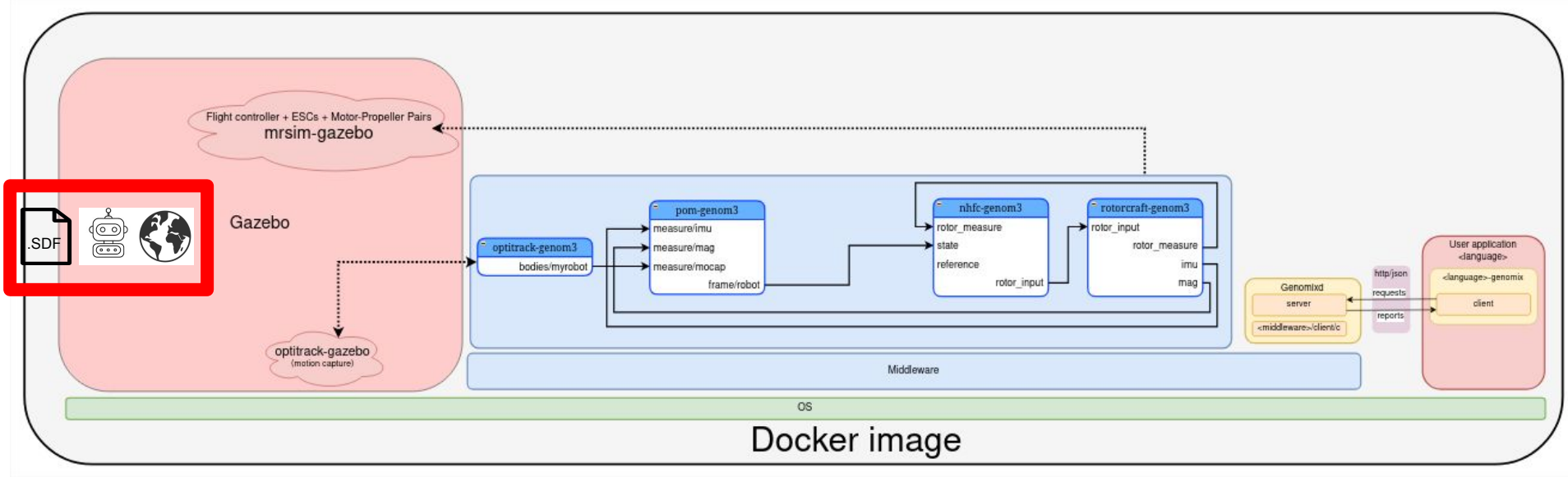
I/O interfaces between simulator and controller (nhfc)



where

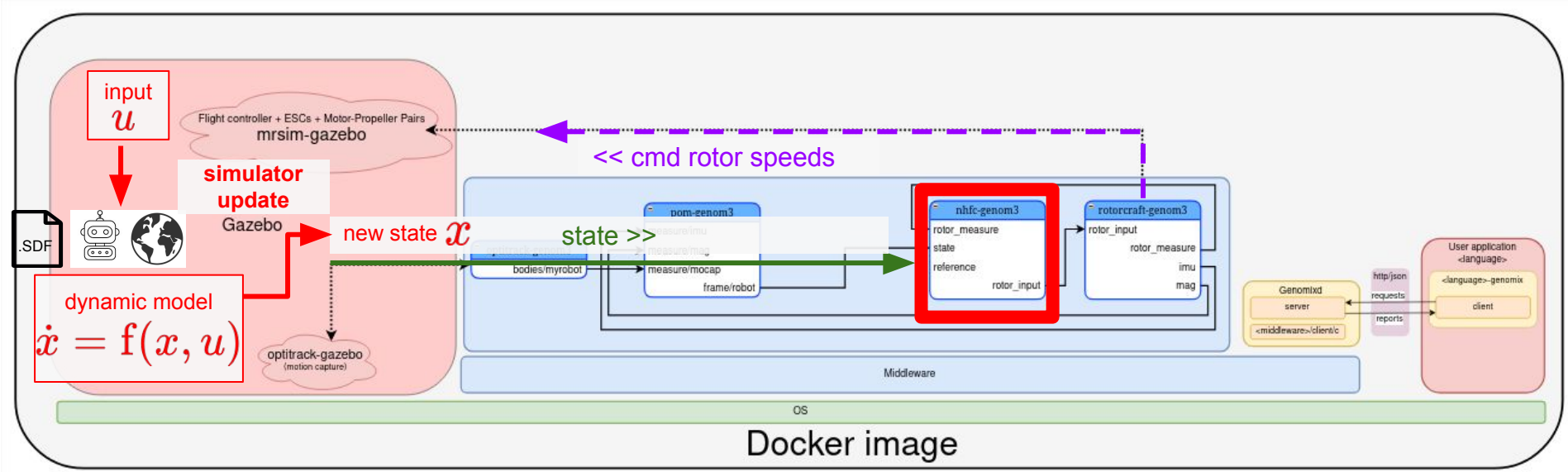
- *<middleware> = pocoLibs*
- *<language> = python (python-genomix documentation)*

telekyb3 – Quadrotor simulation in Gazebo



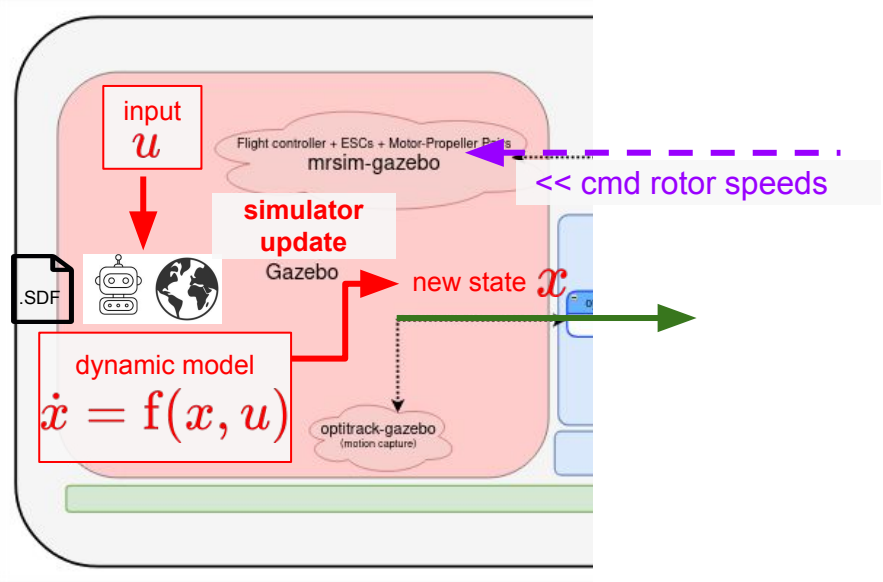
where
- *<middleware> = [pocoLibs]*
- *<language> = python ([python-genomix documentation])*

telekyb3 – Quadrotor simulation in Gazebo



where
- *<middleware> = pocoLibs*
- *<language> = python (python-genomix documentation)*

telekyb3 – Quadrotor simulation in Gazebo



where
- <middleware> = pocoLibs
- <language> = python (python-genomix documentation)

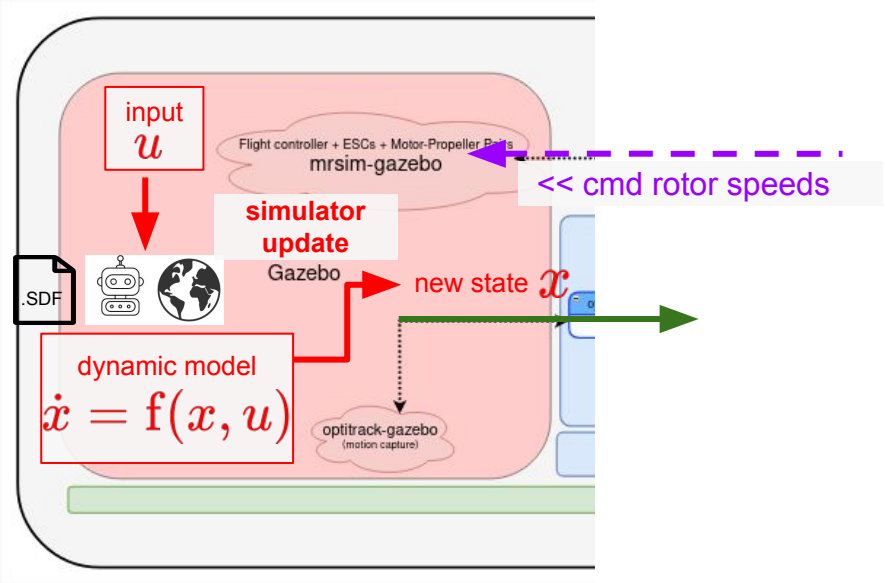
Gazebo simulator engine performs the following steps:

1. Receive a new input
2. Update dynamic model
   a. Model integration
3. Compute new state

However:

- Dynamic model is in **continuous-time**!
- Software implementation simpler in **discrete-time**!

Assignment



where
- <middleware> = pocoLibs
- <language> = python (python-genomix documentation)

Replicate in a Python script the steps performed by the Gazebo simulator engine

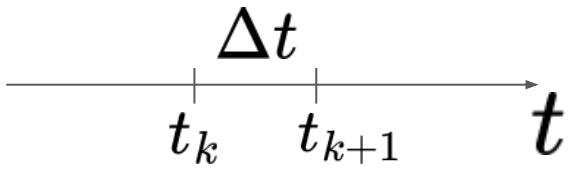
Precisely:

1. Receive a new input
2. Update dynamic model
   a. Model integration
3. Compute new state

At the moment: Ignore telekyb3!

- Create an independent Python script!
- No python-genomix
- No GenoM3 components

Assignment – Open issues



Dynamic model **f(x,u)** is in **continuous-time**!

Software implementation is easier in **discrete-time**:

- Time discretization: equidistant time grid $t_k = k\,\Delta t, \quad \Delta t$ sampling time
- Initial condition: (x_0, u_0)
- At each new time step t_k :
  - Get input at t_k , u_k
  - Compute state at t_{k+1} , x_{k+1}

#1. Which is f(x,u)?

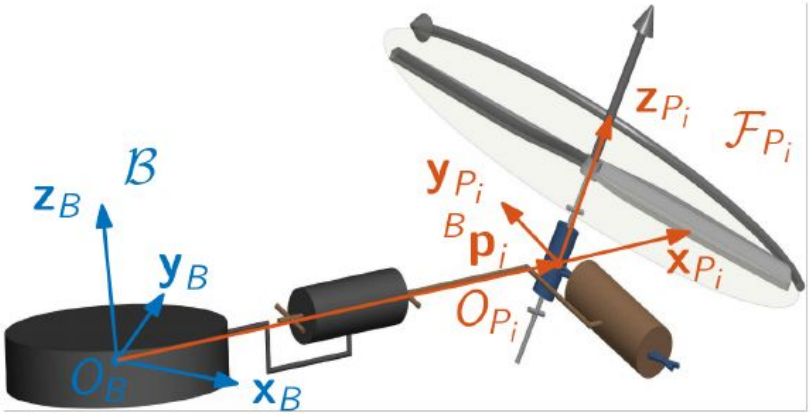
#2. How to get x_{k+1} from continuous-time function f(x,u)?

Robot state

- $\mathcal{W}$: world frame centered in O_W with axes x_W, y_W, z_W, follows the East-North-Up (ENU) convention.

Robot state

- ${}^{W}\boldsymbol{p}_B \in \mathbb{R}^3$: position w.r.t. $\mathcal{W}$
- ${}^{W}\boldsymbol{R}_B \in \mathrm{SO}(3)$: orientation w.r.t. $\mathcal{W}$
- ${}^{W}\dot{\boldsymbol{p}}_B \in \mathbb{R}^3$: linear velocity of the origin of $\mathcal{B}$ expressed in $\mathcal{W}$
- ${}^{B}\boldsymbol{\omega}_B \in \mathbb{R}^3$: angular velocity of $\mathcal{B}$ w.r.t. $\mathcal{W}$, expressed in $\mathcal{B}$



$$\text{state vector } x \longrightarrow \dot{x}? \longrightarrow \dot{x} = \mathrm{f}(x, u) \longrightarrow \begin{matrix} u? \\ \mathrm{f}? \end{matrix}$$

Atomic actuation units (AAUs)

Definition of main variables

For the sake of compactness

- ${}^{B}\boldsymbol{P} = [{}^{B}\boldsymbol{p}_1 \cdots {}^{B}\boldsymbol{p}_N] \in \mathbb{R}^{3\times N}$: concatenation matrices of all AAUs positions
- $\boldsymbol{V} = [\boldsymbol{v}_1 \cdots \boldsymbol{v}_N] \in \mathbb{R}^{3\times N}$: concatenation matrices of all spinning axes
- $\dot{\boldsymbol{V}} = [\dot{\boldsymbol{v}}_1 \ldots \dot{\boldsymbol{v}}_N] \in \mathbb{R}^{3\times N}$: time derivative of $\boldsymbol{V}$
- $\boldsymbol{u}_\lambda = [u_{\lambda_1} \ldots u_{\lambda_N}]^\top \in \mathbb{U}_\lambda \subset \mathbb{R}^{n_\lambda}$: vector gathering the control inputs relative to the thrust intensity. Notice that $\mathbb{U}_\lambda = \times_{i=1}^{N} \mathbb{U}_{\lambda_i}$, $n_\lambda = N$
- $\boldsymbol{u}_{\boldsymbol{V}} = [u_1 \ldots u_{n_m}]^\top \in \mathbb{U}_{\boldsymbol{V}} = \times_{j=1}^{n_m} \mathbb{U}_{\boldsymbol{v}_j} \subset \mathbb{R}^{n_m}$ gathers the angular positions of the shafts of the servo motors that control the orientation of each $\boldsymbol{v}_i$
- $\boldsymbol{u} = [\boldsymbol{u}_\lambda^\top \ \boldsymbol{u}_{\boldsymbol{V}}^\top]^\top \in \mathbb{U} \subset \mathbb{R}^{n_u}$: total control input vector

input vector u

Collective wrench

Force and moment applied to the platform at O_B expressed in $\mathcal{B}$

- $\boldsymbol{f} \in \mathbb{R}^3$: total thrust
- $\boldsymbol{m} \in \mathbb{R}^3$: total moment
- $\boldsymbol{w} \in \mathbb{R}^6$: actuation wrench
- $\mathbb{W}$: set of feasible wrenches

Definition: inputs for the i-th AAU

$$u_{\lambda_i} = |w_i| w_i \in \mathbb{U}_{\lambda_i} \subset \mathbb{R}$$

Force and moment at O_B:

$$\boldsymbol{f}_i = c_{f_i} \boldsymbol{v}_i u_{\lambda_i}$$
$$\boldsymbol{m}_i = k_i c_{\tau_i} \boldsymbol{v}_i u_{\lambda_i} + {}^{B}\boldsymbol{p}_i \times \boldsymbol{f}_i$$

$$\boldsymbol{w}(\boldsymbol{u}) = \begin{bmatrix} \boldsymbol{f}(\boldsymbol{u}) \\ \boldsymbol{m}(\boldsymbol{u}) \end{bmatrix} = \sum_{i=1}^{N} \begin{bmatrix} \boldsymbol{f}_i(\boldsymbol{u}) \\ \boldsymbol{m}_i(\boldsymbol{u}) \end{bmatrix} = \sum_{i=1}^{N} \begin{bmatrix} c_{f_i} \boldsymbol{v}_i u_{\lambda_i} \\ k_i c_{\tau_i} \boldsymbol{v}_i u_{\lambda_i} + {}^{B}\boldsymbol{p}_i \times \boldsymbol{f}_i \end{bmatrix}$$

Aerodynamic parameters

Force and drag moment at $\mathcal{F}_{P_i}$

Aerodynamic parameters (depending on the propeller shape)

- $c_{f_i} \in \mathbb{R}_{>0}$: lift coefficient
- $c_{\tau_i} \in \mathbb{R}_{>0}$: drug coefficient
- $k_i \in \{-1, 1\}$: direction of rotation
  - $k_i = -1$ if the thrust has the same direction of $\boldsymbol{v}_i$ (CCW prop.)
  - $k_i = 1$ if the thrust has the opposite direction of $\boldsymbol{v}_i$ (CW prop.)

$$\boldsymbol{f}_{P_i} = c_{f_i} |w_i| w_i \boldsymbol{v}_i$$
$$\boldsymbol{m}_{P_i} = k_i c_{\tau_i} |w_i| w_i \boldsymbol{v}_i,$$

Thrust direction

Unidirectional or bidirectional thrust

2-nd parameter: **direction of the thrust** along the spinning axis

- *unidirectional thrust* (most common): the brushless motor controllers rotate the propellers only in one direction,

$$0 \leq \underline{w}_i \leq w_i \leq \bar{w}_i$$

- *bidirectional thrust* (less common): the brushless motor controllers rotate the propellers in both directions,

$$\underline{w}_i \leq w_i \leq \bar{w}_i \text{ where } \underline{w}_i \in \mathbb{R}_{<0} \text{ and } \bar{w}_i \in \mathbb{R}_{>0}$$

Thrust re-orientation

3-rd parameter: ability to **re-orient its thrust**

- *fixed spinning axes*: $\boldsymbol{v}_i$ is constant
- *actuated spinning axes*: $\boldsymbol{v}_i$ can be changed actively or passively

$$\boldsymbol{v}_i = f_{\boldsymbol{v}_i}(\boldsymbol{u}_{\boldsymbol{V}})$$

  - $f_{\boldsymbol{v}_i} : \mathbb{R}^{n_m} \to \mathbb{S}^2$
  - $\boldsymbol{u}_{\boldsymbol{V}} = [u_1 \dots u_{n_m}]^\top \in \mathbb{U}_{\boldsymbol{V}} = \times_{j=1}^{n_m} \mathbb{U}_{\boldsymbol{v}_j} \subset \mathbb{R}^{n_m}$ gathers the angular positions of the shafts of the servo motors that control the orientation of each $\boldsymbol{v}_i$
  - n_m is the number of servomotors. Each motor is assumed to be tilted with a maximum number of 2 servomotors, which in turns implies that $n_m \leq 2N$

Equations of motion

Using the Newton-Euler formalism: (floating body dynamics)

$$\begin{bmatrix} m\,{}^{W}\ddot{\boldsymbol{p}}_{B} \\ \boldsymbol{J}\,{}^{B}\dot{\boldsymbol{\omega}}_{B} \end{bmatrix} = - \begin{bmatrix} mgz_{W} \\ {}^{B}\boldsymbol{\omega}_{B} \times \boldsymbol{J}\,{}^{B}\boldsymbol{\omega}_{B} \end{bmatrix} + \mathbf{G}\boldsymbol{w}(\boldsymbol{u}), \quad \approx \dot{x} = \tilde{\mathrm{f}}(x,u)$$

- $m \in \mathbb{R}_{>0}$ and $\boldsymbol{J} \in \mathbb{R}^{3\times3}_{>0}$ are the total mass and the inertia matrix w.r.t. $\mathcal{B}$
- g is the gravitational acceleration constant
- $\mathbf{G}$ rotates the applied wrench (and more specifically $\boldsymbol{f}$)

$$\mathbf{G} = \begin{bmatrix} {}^{W}\boldsymbol{R}_{B} & \mathbf{0}_{3\times3} \\ \mathbf{0}_{3\times3} & \mathbf{I}_{3\times3} \end{bmatrix},$$

State-space formulation $\dot{x} = \mathrm{f}(x, u)$

$$\begin{bmatrix} m & {}^{W}\ddot{\mathbf{p}}_{B} \\ \mathbf{J} & {}^{B}\dot{\boldsymbol{\omega}}_{B} \end{bmatrix} = - \begin{bmatrix} mg\ \mathbf{z}_{W} \\ {}^{B}\boldsymbol{\omega}_{B} \times \mathbf{J}\ {}^{B}\boldsymbol{\omega}_{B} \end{bmatrix} + \mathbf{G}\ \mathbf{w}(\mathbf{u})$$

$$\begin{bmatrix} m\mathbf{I}_3 & \mathbf{0}_3 \\ \mathbf{0}_3 & \mathbf{J} \end{bmatrix} \begin{bmatrix} {}^{W}\ddot{\mathbf{p}}_{B} \\ {}^{B}\dot{\boldsymbol{\omega}}_{B} \end{bmatrix} = - \begin{bmatrix} mg\ \mathbf{z}_{W} \\ {}^{B}\boldsymbol{\omega}_{B} \times \mathbf{J}\ {}^{B}\boldsymbol{\omega}_{B} \end{bmatrix} + \mathbf{G}\ \mathbf{w}(\mathbf{u})$$

$$\begin{bmatrix} {}^{W}\ddot{\mathbf{p}}_{B} \\ {}^{B}\dot{\boldsymbol{\omega}}_{B} \end{bmatrix} = \begin{bmatrix} m\mathbf{I}_3 & \mathbf{0}_3 \\ \mathbf{0}_3 & \mathbf{J} \end{bmatrix}^{-1} \left(- \begin{bmatrix} mg\ \mathbf{z}_{W} \\ {}^{B}\boldsymbol{\omega}_{B} \times \mathbf{J}\ {}^{B}\boldsymbol{\omega}_{B} \end{bmatrix} + \mathbf{G}\ \mathbf{w}(\mathbf{u}) \right)$$

$$\begin{bmatrix} {}^{W}\ddot{\mathbf{p}}_{B} \\ {}^{B}\dot{\boldsymbol{\omega}}_{B} \end{bmatrix} = \begin{bmatrix} \frac{1}{m}\mathbf{I}_3 & \mathbf{0}_3 \\ \mathbf{0}_3 & \begin{bmatrix} \frac{1}{J_{xx}} & 0 & 0 \\ 0 & \frac{1}{J_{yy}} & 0 \\ 0 & 0 & \frac{1}{J_{zz}} \end{bmatrix} \end{bmatrix} \left(- \begin{bmatrix} mg\ \mathbf{z}_{W} \\ {}^{B}\boldsymbol{\omega}_{B} \times \mathbf{J}\ {}^{B}\boldsymbol{\omega}_{B} \end{bmatrix} + \mathbf{G}\ \mathbf{w}(\mathbf{u}) \right)$$

assumption: diagonal inertial matrix

State-space formulation

$$\mathbf{x} = \begin{bmatrix} {}^{W}\mathbf{p}_{B}{}^{T} & {}^{W}\mathbf{q}_{B}{}^{T} & {}^{B}\mathbf{v}_{B}{}^{T} & {}^{B}\boldsymbol{\omega}_{B}{}^{T} \end{bmatrix}^{T}$$

(1 2 3 | 4 5 6 7 | 8 9 10 | 11 12 13)

$$\mathbf{u} = \mathbf{u}_{\lambda}$$

scalar first!

$$\dot{\mathbf{x}} = \begin{bmatrix} {}^{W}\dot{\mathbf{p}}_{B} \\ {}^{W}\dot{\mathbf{q}}_{B} \\ {}^{W}\dot{\mathbf{v}}_{B} \\ {}^{B}\dot{\boldsymbol{\omega}}_{B} \end{bmatrix} = \begin{bmatrix} {}^{W}\dot{\mathbf{p}}_{B} \\ {}^{W}\dot{\mathbf{q}}_{B} \\ {}^{W}\ddot{\mathbf{p}}_{B} \\ {}^{B}\dot{\boldsymbol{\omega}}_{B} \end{bmatrix} = \begin{bmatrix} \mathbf{x}_{8:10} \\ ? \\ \tilde{\mathbf{f}}_{1:3} \\ \tilde{\mathbf{f}}_{4:6} \end{bmatrix} = \begin{bmatrix} \mathbf{x}_{8:10} \\ \frac{1}{2}\begin{bmatrix} 0 \\ {}^{W}\boldsymbol{\omega}_{B} \end{bmatrix} \otimes {}^{W}\mathbf{q}_{B} \\ \tilde{\mathbf{f}}_{1:3} \\ \tilde{\mathbf{f}}_{4:6} \end{bmatrix} = \begin{bmatrix} \mathbf{x}_{8:10} \\ (\mathbf{x}_{11:13}, \mathbf{x}_{4:7}) \\ \tilde{\mathbf{f}}_{1:3} \\ \tilde{\mathbf{f}}_{4:6} \end{bmatrix} = \mathbf{f}(\mathbf{x}, \mathbf{u})$$

??

#1

Hamiltonian product of two quaternions:
rotation composition using quaternions

Hamilton product of two quaternions

Definition:

For two quaternions

$$q = (s_1, \mathbf{v}_1), \quad p = (s_2, \mathbf{v}_2),$$

the Hamilton product is

$$q \otimes p = \left(s_1 s_2 - \mathbf{v}_1 \cdot \mathbf{v}_2, \quad s_1 \mathbf{v}_2 + s_2 \mathbf{v}_1 + \mathbf{v}_1 \times \mathbf{v}_2\right).$$

- $\cdot$: dot product
- $\times$: cross product

Hamilton product of two quaternions

Computation Example

Let

$$q_1 = (a_1, b_1, c_1, d_1), \quad q_2 = (a_2, b_2, c_2, d_2).$$

Then $q_1 \otimes q_2 = (w, x, y, z)$ with:

$$\begin{aligned} w &= a_1a_2 - b_1b_2 - c_1c_2 - d_1d_2 \\ x &= a_1b_2 + b_1a_2 + c_1d_2 - d_1c_2 \\ y &= a_1c_2 - b_1d_2 + c_1a_2 + d_1b_2 \\ z &= a_1d_2 + b_1c_2 - c_1b_2 + d_1a_2 \end{aligned}$$

From continuous to discrete time

Continuous-time system: $\dot{x}(t) = f(x(t), u(t))$

$x(t)$: state
$u(t)$: input

Introduce Time discretization: $t_k = k\Delta t$

Define: $x_k := x(t_k), \quad u_k := u(t_k)$

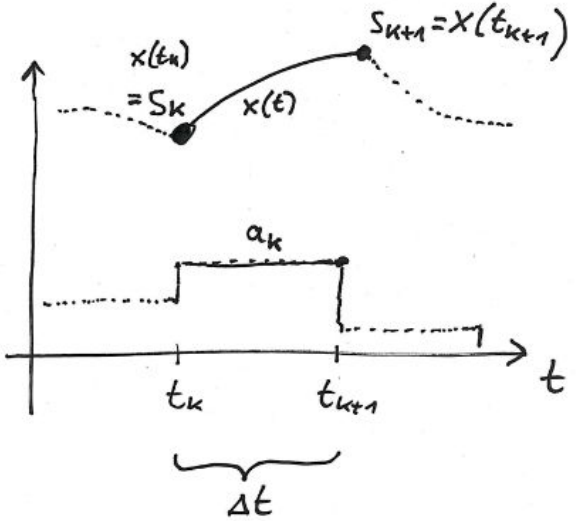
where the input u is kept constant over Δt (zero-order holder)

Discrete-time system: $x_{k+1} = f_d(x_k, u_k)$

where the discrete-time dynamics is obtained via numerical integration

Reference: *J. Boedecker and M. Diehl, University Freiburg, Model Predictive Control and Reinforcement Learning – Lecture 2: Dynamic Systems and Simulation.*

Numerical integration



Using forward Euler approximation:

$$\dot{x}(t) \approx \frac{x_{k+1} - x_k}{\Delta t}$$

we obtain:

$$x_{k+1} = x_k + \Delta t\, f(x_k, u_k)$$

Reference: *J. Boedecker and M. Diehl, University Freiburg, Model Predictive Control and Reinforcement Learning – Lecture 2: Dynamic Systems and Simulation.*

Numerical integration

Using forward Euler approximation:

$$\dot{x}(t) \approx \frac{x_{k+1} - x_k}{\Delta t}$$

we obtain:

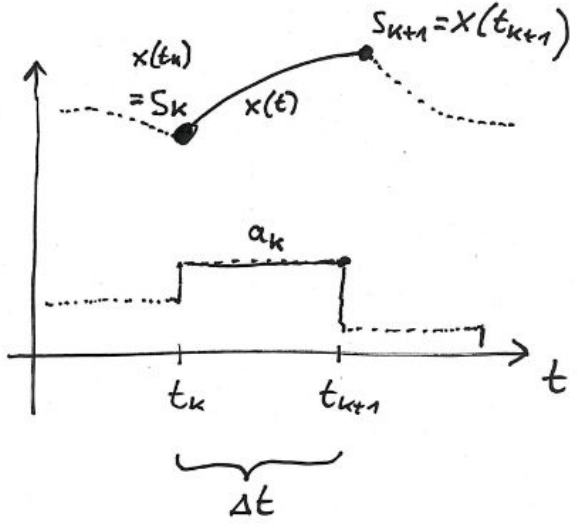
$$x_{k+1} = x_k + \Delta t\, f(x_k, u_k)$$

Considerations:

- Small Δt:
  - better approximation
  - higher computational cost
- Large Δt
  - faster
  - less accurate
- Interpretation: first-order approximation
  - Not very accurate method, as it may result in large errors!

Reference: *J. Boedecker and M. Diehl, University Freiburg, Model Predictive Control and Reinforcement Learning – Lecture 2: Dynamic Systems and Simulation.*

Numerical integration



Better approach is using Runge Kutta (RK) methods

Intermediate updates:

$$k_1 = f(x_k, u_k)$$

$$k_2 = f\left(x_k + \frac{\Delta t}{2} k_1,\, u_k\right)$$

$$k_3 = f\left(x_k + \frac{\Delta t}{2} k_2,\, u_k\right)$$

$$k_4 = f\left(x_k + \Delta t\, k_3,\, u_k\right)$$

Then:

$$x_{k+1} = x_k + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$ **RK 4-th order**

#2

Reference: *J. Boedecker and M. Diehl, University Freiburg, Model Predictive Control and Reinforcement Learning – Lecture 2: Dynamic Systems and Simulation.*

New problem – Integrate quaternions

A quaternion representing a rotation must satisfy: $\|q\| = 1$

But when you integrate the dynamics: $\dot{q} = \frac{1}{2} q \otimes \omega$

standard numerical methods (Euler, RK4, etc.) do not preserve this constraint.

Solution?

Normalize quaternion after integration

Solution to previous problem – Normalization quaternions

Given a quaternion: $$q = (w, x, y, z)$$

Compute its norm as: $$\|q\| = \sqrt{w^2 + x^2 + y^2 + z^2}$$

Then, normalize it: $$q_{\text{normalized}} = \left(\frac{w}{\|q\|}, \frac{x}{\|q\|}, \frac{y}{\|q\|}, \frac{z}{\|q\|} \right)$$

Another problem and its solution

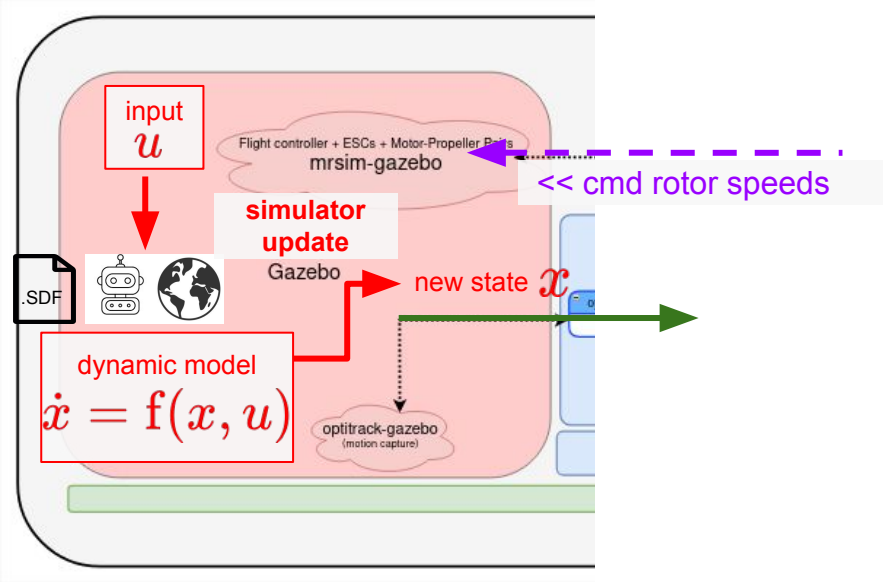
In Gazebo simulation, there is the ground!

How to account for ground reaction?

If the robot height is below the ground then:

- Keep constant position and attitude (z = ground level)
- Reset all velocities and accelerations (set them equal to 0)

Assignment - Recap



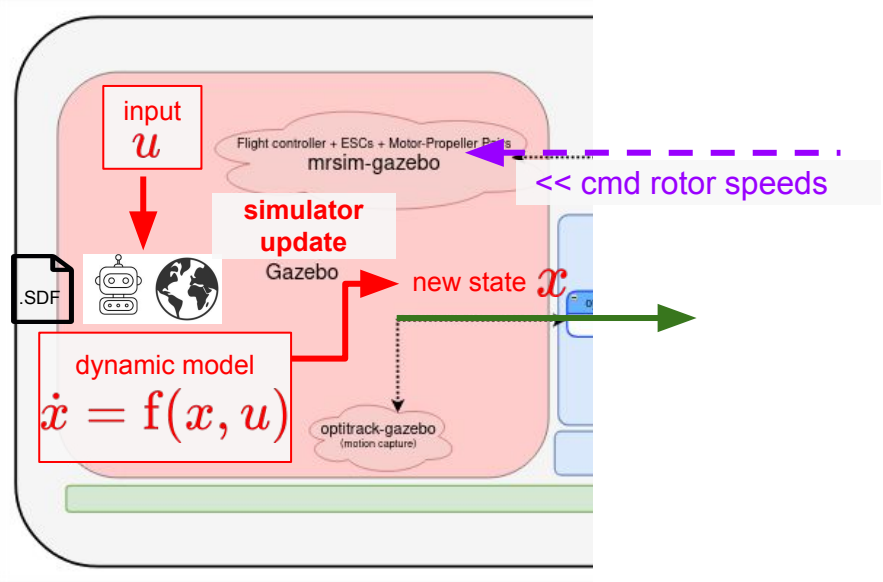
where
- <middleware> = pocoLibs
- <language> = python (python-genomix documentation)

Replicate in a Python script the steps performed by the Gazebo simulator engine

Precisely:

1. Initialize simulation with initial state and input conditions
2. Repeat at every time instant $t_k = k\Delta t$:
   a. Receive a new discrete-time input
   b. Integration of dynamic model with RK4
   c. Normalize quaternion
   d. Compute state at new time instant
   e. Check robot height:
      if necessary, then apply ground reaction

Assignment - Recap



Considerations:

- Create an independent Python script!
- Define Δt = 1ms
- Test simulator with
  - lifting force equal to weight force
  - lifting force 1N above weight force
- No python-genomix
- No GenoM3 components
- Generate plots as yesterday assignment
  - No controller setpoints here!
  - Only measured (current) robot state!

where
- <middleware> = pocoLibs
- <language> = python (python-genomix documentation)

Numpy basics

- NumPy fundamentals → https://numpy.org/doc/stable/user/basics.html
- NumPy for MATLAB users → https://numpy.org/doc/stable/user/numpy-for-matlab-users.html#numpy-for-matlab-users
- NB: (rows,) is not (rows, 1) !
  - (rows,) is 1D vector
  - (rows, 1) is an 2D vector, thus a matrix!
  - Use (rows,) for column vectors

Numpy basics

```
>>> import numpy as np
>>> a = np.array([1, 2, 3])
>>> a.shape
(3,)
>>> b = np.array([4, 5, 6])
>>> b.shape
(3,)
```

```
>>> c = np.hstack((a, b))
>>> c
array([1, 2, 3, 4, 5, 6])
>>> c.shape()
(6,)
```

Numpy basics

```
>>> d = np.array([[1, 2, 3]])
>>> d
array([[1, 2, 3]])
>>> d.shape
(1, 3)
```

```
>>> e = d.transpose()
>>> e
array([[1],
       [2],
       [3]])
>>> e.shape
(3, 1)
# which is not (3,)!
# e is a 2D vector (a matrix) and
not 1D vector
```